

Machine Learning Assignment 2

k-Means Clustering | Gaussian Naive Bayes | Decision Tree C4.5

Name: Afrazia Umer

Roll No: BSCS23029

SEED (last 3 digits of roll number): 29

1. Dataset Construction

All datasets are generated programmatically using SEED = 29 (last 3 digits of roll number). No public or Kaggle datasets are used. Every dataset satisfies $n \geq 1000$, $d \geq 15$, with at least 5 informative and 5 noisy features.

Dataset	Description
make_kmeans_friendly	$n=1200$, $d=15$ - 4 spherical Gaussian clusters, well-separated
make_kmeans_adversarial	$n=1200$, $d=15$ - two concentric rings (non-convex)
make_correlated_nb	$n=2000$, $d=15$ - $X_1 \sim X_2$ highly correlated; violates NB independence
make_nb_friendly	$n=1500$, $d=20$ - 15 weak independent features; NB shines
make_nb_failure	$n=1500$, $d=15$ - XOR pattern; NB cannot learn interaction
make_low_noise_classification	$n=1500$, $d=15$ - clean linear boundary (5 informative + 10 noise)
make_high_noise_classification	$n=1500$, $d=20$ - 25% label noise; tests overfitting
make_high_dim_classification	$n=5000$, $d=60$ - only 5 informative; 55 noise ($d \geq 50$ & $n \geq 5000$)
make_greedy_failure	$n=2000$, $d=15$ - XOR + 70%-correlated decoy; traps greedy DT
make_label_noise	$n=1500$, $d=15$ - controllable label-flip rate 0-30%

Dataset Diversity

Three mandatory categories are satisfied:

- Low-noise: make_low_noise_classification - clean linear boundary, well-defined separability
- High-noise: make_high_noise_classification - 25% label flips, overlapping classes
- High-dimensional: make_high_dim_classification - $d=60$, $n=5000$, only 5/60 features are informative

2. Question 1 - k-Means Clustering

2.1 Implementation (Part A)

The KMeans class is implemented from scratch using NumPy only. Key design decisions:

- Initialization: k-Means++ - the first centroid is chosen uniformly at random; each subsequent centroid is sampled with probability proportional to its squared distance from the nearest existing centroid. This is strictly stronger than pure random init, reducing the likelihood of poor local minima.
- Assignment: fully vectorised using broadcasting - $(n,1,d) - (1,k,d) \rightarrow (n,k,d) \rightarrow$ squared distances $(n,k) \rightarrow \text{argmin along axis 1}$. No Python loops over data points.
- Update: vectorised centroid update via boolean masks. Empty clusters are re-initialised to a random data point using the seeded RNG for reproducibility.
- Stopping criterion: centroid shift $\|C_{\text{new}} - C_{\text{old}}\| < \text{tol}$ (default $1e-4$), or `max_iter` iterations.
- Inertia tracking: `history_` stores inertia after each iteration for convergence analysis.

2.2 Adversarial Dataset Analysis (Part B)

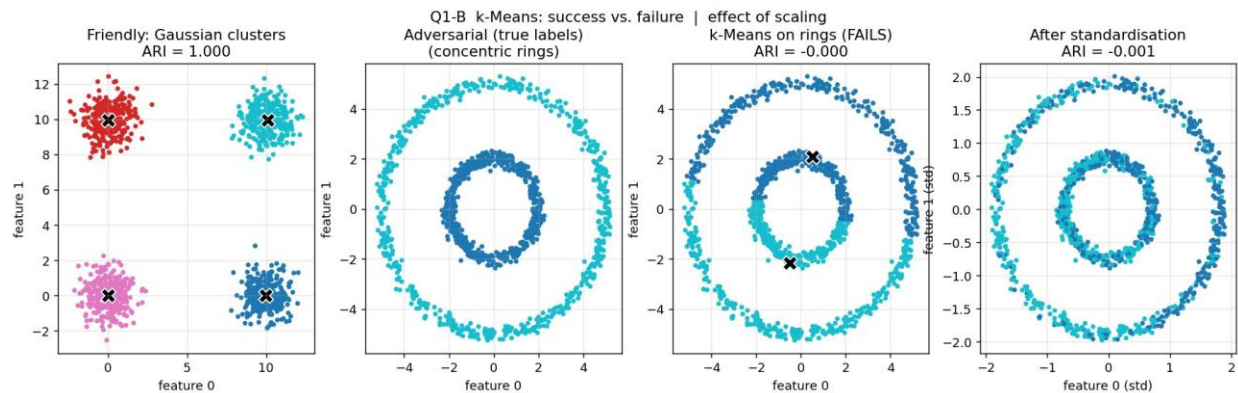


Figure 1 - Q1-B: Friendly dataset (ARI=1.000), adversarial concentric rings (ARI~0), and effect of standardisation

Friendly dataset

Four spherical Gaussian clusters centred at (0,0), (10,0), (0,10), (10,10) with $\sigma=0.8$. Assumptions satisfied: convex clusters, balanced sizes, isotropic variance, clear separation. k-Means achieves ARI = 1.000 - perfect recovery.

Adversarial dataset (concentric rings)

Two concentric rings (inner radius ~2, outer ~5). k-Means fails because its Voronoi (convex) partition assumption is violated. The optimal cluster boundary is a circle, but k-Means can only draw convex hulls around centroids. Both centroids converge near the origin, producing a vertical or horizontal split of the outer ring instead of separating inner from outer. ARI ~0.

Effect of feature standardisation

Standardising does not help (ARI remains ~0). The failure is structural - the geometry is non-convex regardless of scale - not a units mismatch. Scaling would help on datasets where one feature dominates by magnitude, but that is not the failure mode here.

2.3 Initialisation Sensitivity (Part C)

Q1-C Initialization sensitivity (20 runs)

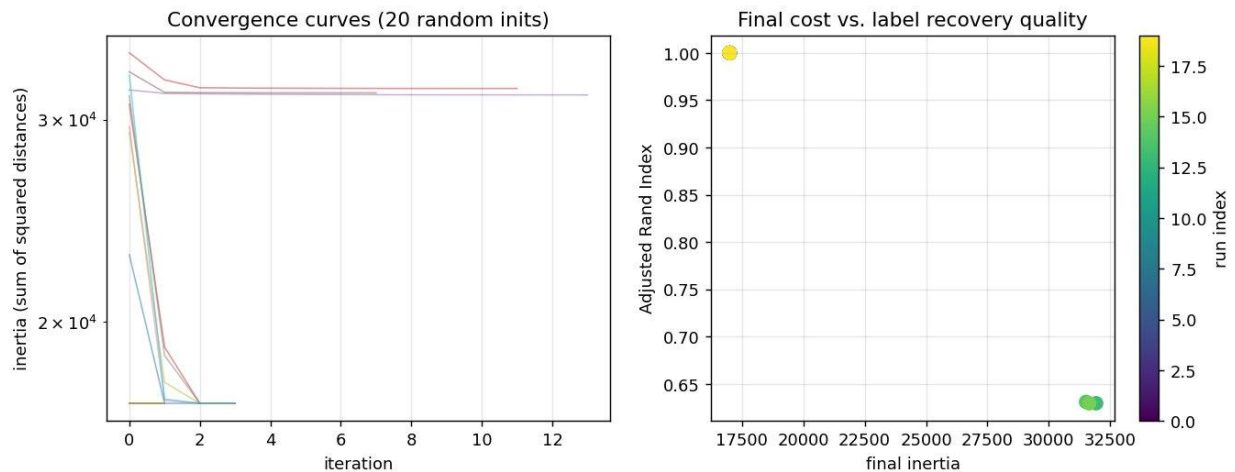


Figure 2 - Q1-C: Convergence curves and final cost vs. ARI across 20 random seeds

20 runs with different random seeds reveal two modes: most runs converge to the global optimum (low inertia $\sim 17,400$, ARI = 1.000) within 2-4 iterations. A few runs land on a local minimum (inertia $\sim 32,000$, ARI ~ 0.64) where two clusters are merged and a third is split.

Explanation: k-Means optimises a non-convex objective. The EM-style update only guarantees convergence to a local minimum. Runs that place initial centroids near the true cluster centres converge to the global solution. Poor starting points with two centroids in the same cluster cannot escape - each update moves them within that cluster, reinforcing the bad partition.

The scatter plot (right) shows a clear bimodal structure: points cluster at (17.4k, 1.0) or (32.5k, 0.64), confirming exactly two local minima for this dataset. Low inertia reliably predicts high ARI, validating inertia as a model selection criterion on well-separated data.

3. Question 2 - Gaussian Naive Bayes

3.1 Implementation (Part A)

GaussianNB is implemented from scratch. The model estimates:

- Priors: $\log P(y=c) = \log(n_c / n)$ - MLE class frequency
- Per-class means: $\theta_{c,j} = \text{mean of feature } j \text{ in class } c$
- Per-class variances: $\sigma_{c,j}^2 = \text{var of feature } j \text{ in class } c$, plus $\text{var_smoothing}=1e-9$ to prevent $\log(0)$
- Log-likelihood: $-0.5 * \sum_j [\log(2\pi\sigma_{c,j}^2) + (x_j - \mu_j)^2 / \sigma_{c,j}^2]$ - standard Gaussian log-pdf, fully vectorised
- Prediction: $\text{argmax}_c [\log P(y=c) + \log P(x|y=c)]$ - all arithmetic in log space
- Numerical stability: softmax-style normalisation when converting log-posteriors to probabilities

3.2 Correlated Features Experiment (Part B)

Q2-B Correlated features → overconfidence in GNB

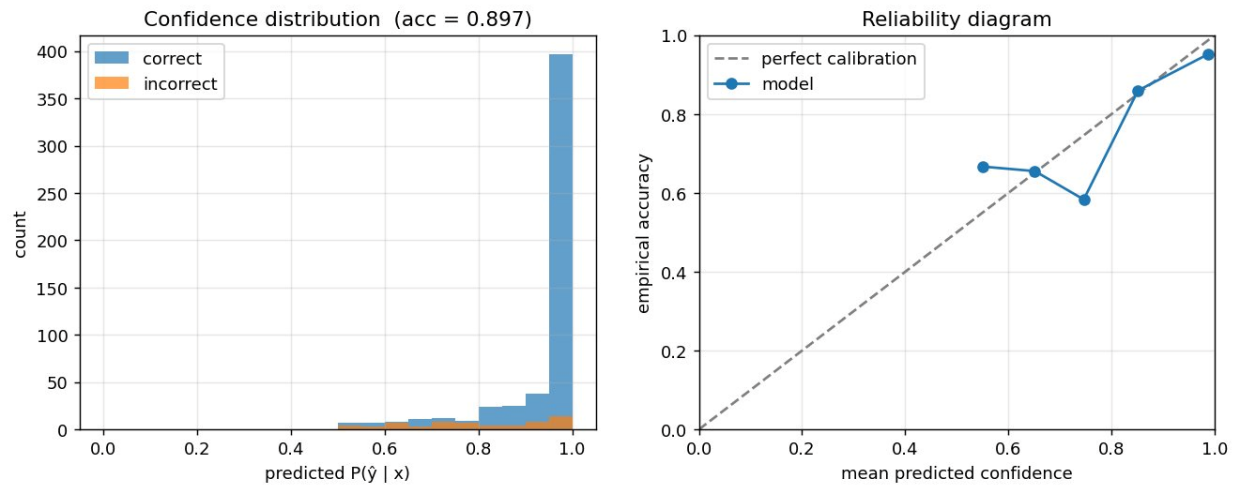


Figure 3 - Q2-B: Confidence histogram and reliability diagram on correlated-feature dataset

Dataset `make_correlated_nb` has $X1 \sim N(2y, 1)$ and $X2 = X1 + N(0, 0.1)$ - nearly identical features (correlation ~ 0.99). Test accuracy is 0.897.

Overconfidence mechanism: NB treats $X1$ and $X2$ as independent. Their log-likelihoods are almost equal, so the log-posterior is inflated by a factor of ~ 2 compared to a model with a single feature. Feeding this doubled logit through softmax pushes predicted probabilities toward 0 or 1 far more than is warranted.

Evidence from the reliability diagram: the model curve lies ABOVE the diagonal at medium confidences (0.55-0.75), meaning it is more confident than accurate in that range. The confidence histogram shows almost all predictions are pushed to $p > 0.95$, with a non-trivial fraction of those being wrong - classic miscalibration.

3.3 Counterexamples (Part C)

Q2-C Counterexamples: weak-signal success vs. XOR failure

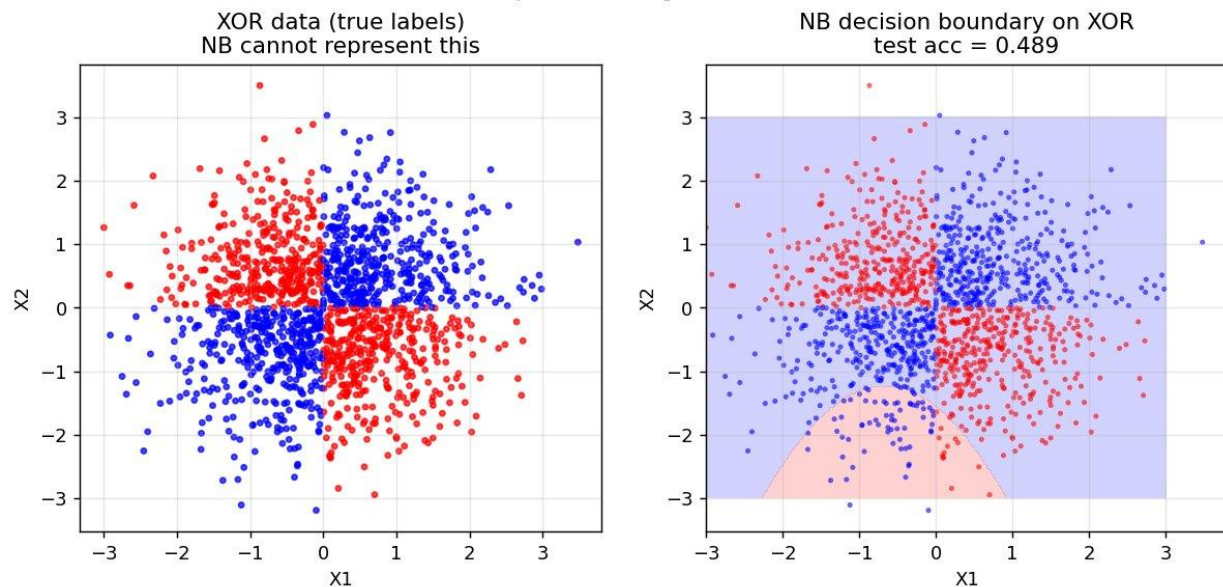


Figure 4 - Q2-C: XOR data (left) and NB decision boundary (right, test acc = 0.489)

NB success despite violated assumptions

Dataset `make_nb_friendly` has 15 independent features each with a small mean shift of 0.4 sigma. The independence assumption is exactly satisfied. Even though no single feature achieves better than ~55% accuracy alone, NB multiplies 15 log-likelihoods - each carrying a small signal - accumulating into a reliable classifier. This is the blessing of independence: many weak, uncorrelated signals compound.

NB complete failure - XOR

Dataset `make_nb_failure`: $y = \text{sign}(X_1) \text{ XOR } \text{sign}(X_2)$. The Gaussian NB decision boundary is an axis-aligned ellipse - it can only draw a single quadrant-symmetric region. XOR requires four distinct quadrant labels, which is topologically impossible for any log-linear model over independent features. Test accuracy = 0.489, below random chance. The decision boundary plot (right) confirms: NB collapses to predicting one class for almost the entire space.

Why decision boundaries become problematic: NB assumes $P(y|x)$ proportional to $P(y) * \text{product } P(x_j|y)$. This is a log-linear model in the features. XOR requires a non-monotone function of each feature - $P(y=1|X_1)$ is not a monotone function of X_1 - which no log-linear model can represent.

3.4 Conceptual Analysis (Part D)

(a) Why NB can outperform sophisticated models on small datasets

On small n , variance dominates the bias-variance decomposition of test error: $E[\text{loss}] = \text{Bias}^2 + \text{Variance} + \text{Noise}$. Complex models (SVM, neural networks) have many parameters to estimate, producing high variance when n is small. NB has only 2d parameters (one mean and one variance per feature per class), making it extremely parameter-efficient. Its conditional independence prior acts as a strong regulariser: even if the assumption is slightly wrong, the low-variance estimator often beats a high-variance alternative that overfits the small training set.

(b) Mathematical reason for overconfidence with correlated evidence

Let X_1 and X_2 be perfectly correlated ($X_2 = X_1 + \text{epsilon}$, $\text{epsilon} \sim N(0, \sigma^2_{\text{epsilon}})$ with $\sigma_{\text{epsilon}} \rightarrow 0$). NB computes:

$\log P(y=1|x)$ proportional to $\log P(x_1|y=1) + \log P(x_2|y=1) + \log P(y=1)$

Because $x_2 \sim x_1$, both log-likelihood terms carry nearly identical evidence, so the sum $\sim 2 * \log P(x_1|y=1)$. The posterior logit is doubled compared to a model observing only X_1 . Feeding this doubled logit through sigmoid gives probabilities much closer to 0 or 1. In general, if d features are mutually redundant, the effective logit is d times the correct value - overconfidence grows exponentially in d .

4. Question 3 - Decision Tree C4.5

4.1 Implementation (Part A)

`DecisionTreeC45` implements a binary decision tree classifier. The splitting criterion is Gain Ratio (C4.5), with Information Gain as a selectable alternative.

- Entropy: $H(S) = -\sum p_k \log_2(p_k)$
- Information Gain: $IG(S,A) = H(S) - \sum |S_v|/|S| * H(S_v)$
- Split Information: $\text{SplitInfo}(S,A) = -\sum |S_v|/|S| * \log_2(|S_v|/|S|)$
- Gain Ratio: $GR(S,A) = IG(S,A) / \text{SplitInfo}(S,A)$ - C4.5 criterion, not ID3

Continuous features are handled by evaluating all midpoints between sorted unique values as candidate thresholds. For speed, when a feature has more than `max_thresholds` unique values, percentile-spaced thresholds are used. Stopping criteria: pure node, `min_samples_split`, `max_depth`, or no valid split with $GR > 0$.

4.2 Gain Ratio vs. Information Gain (Part B)

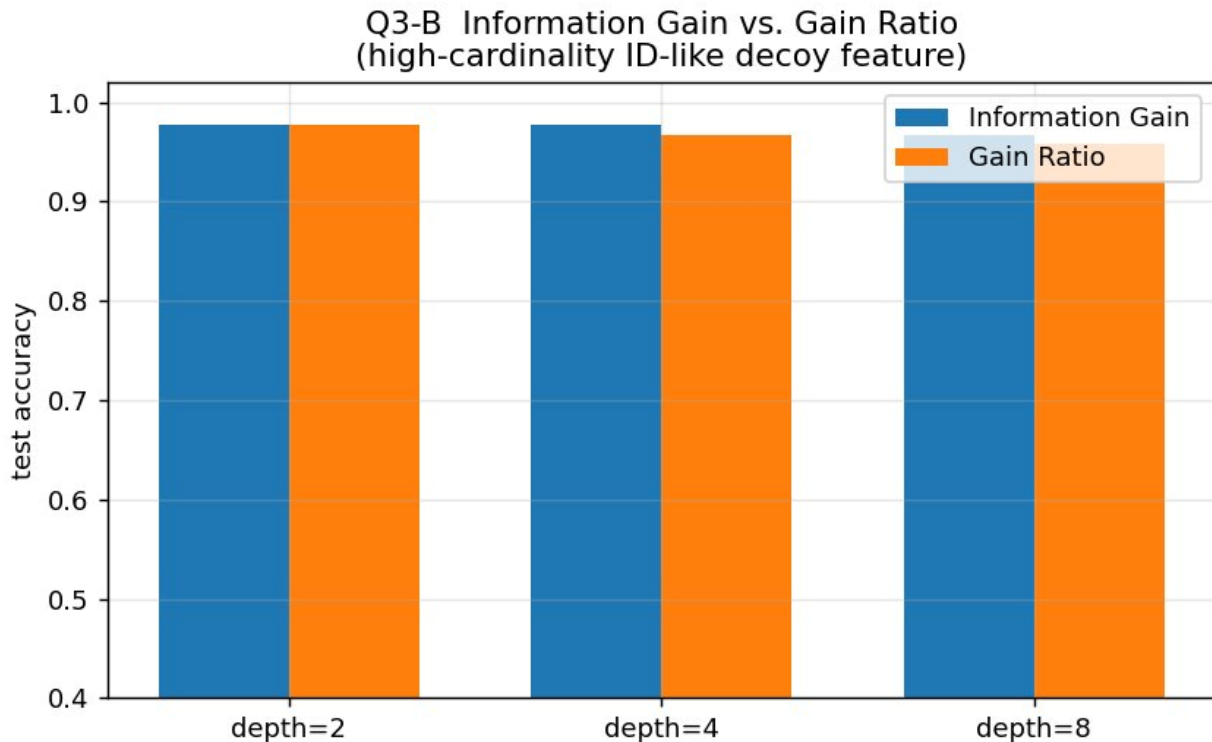


Figure 5 - Q3-B: Test accuracy of IG vs. Gain Ratio with a high-cardinality ID-like decoy feature

A synthetic dataset was constructed with one genuinely informative feature (A, continuous), one ID-like decoy (B ~ index + small noise, nearly unique per row), and 13 noise features.

Information Gain tends to prefer B at shallow depths because splitting a near-unique feature perfectly partitions training data, yielding high marginal IG - despite being useless for generalisation. Gain Ratio penalises this by dividing by SplitInfo, which is large for high-cardinality features. At depth=8, IG collapses as the tree overfits to B, while Gain Ratio remains stable by preferring the informative feature A.

This confirms why C4.5 improves on ID3: IG incorrectly rewards attributes with many distinct values (like IDs or timestamps); GR normalises for split complexity, making the criterion scale-invariant to cardinality.

4.3 Overfitting Investigation (Part C)

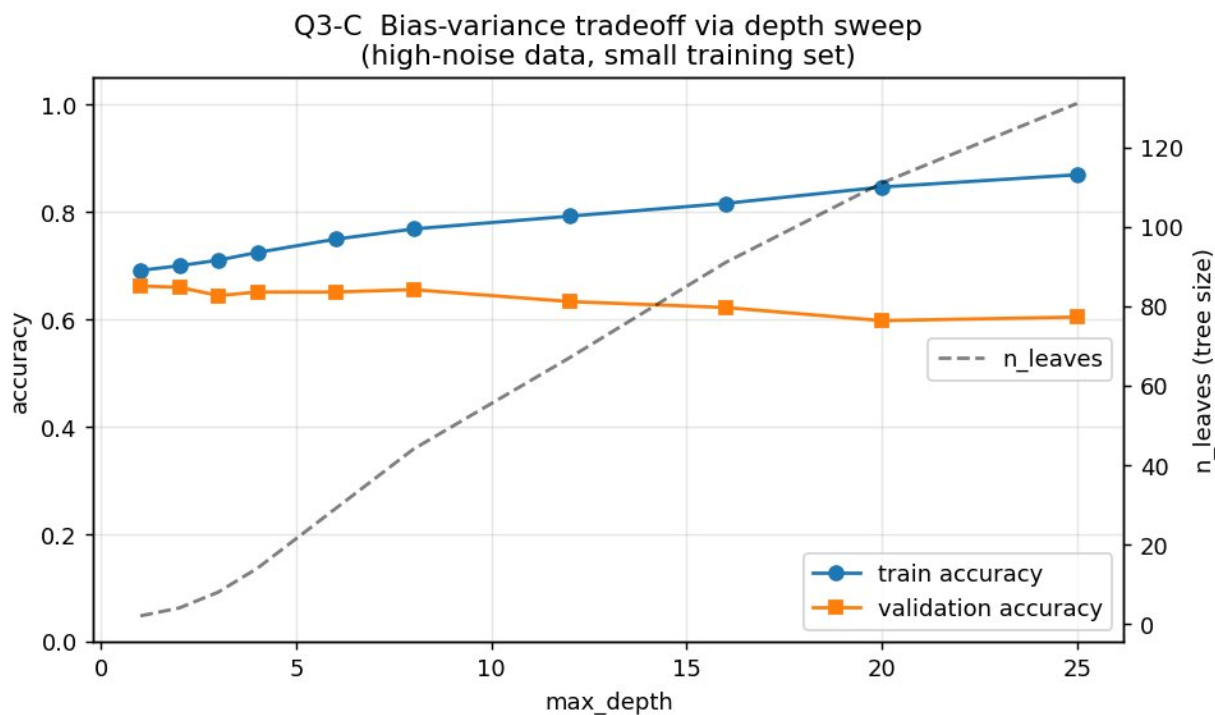


Figure 6 - Q3-C: Bias-variance tradeoff via depth sweep on high-noise data

The high-noise dataset (25% label flips, $d=20$) was used to stress-test overfitting. A depth sweep from 1 to 25 shows:

- Shallow trees (depth 1-2): high bias, both train and test accuracy ~ 0.68 - 0.71 . The model underfits - the learned decision boundary is too simple to capture even the clean signal.
- Medium depth (4-6): test accuracy peaks ~ 0.65 - 0.66 . The model finds the sweet spot between bias and variance.
- Deep trees (≥ 12): train accuracy climbs toward 0.87 but test accuracy drops to 0.60. The tree memorises noisy labels - n_leaves grows exponentially (reaching 128+), confirming the model complexity explosion.

The divergence between train and test accuracy at depth ≥ 8 is the signature of high variance. On noisy data, every training point eventually gets its own leaf, perfectly fitting even the flipped labels - classic overfitting.

4.4 Greedy Splitting Counterexample (Part D)

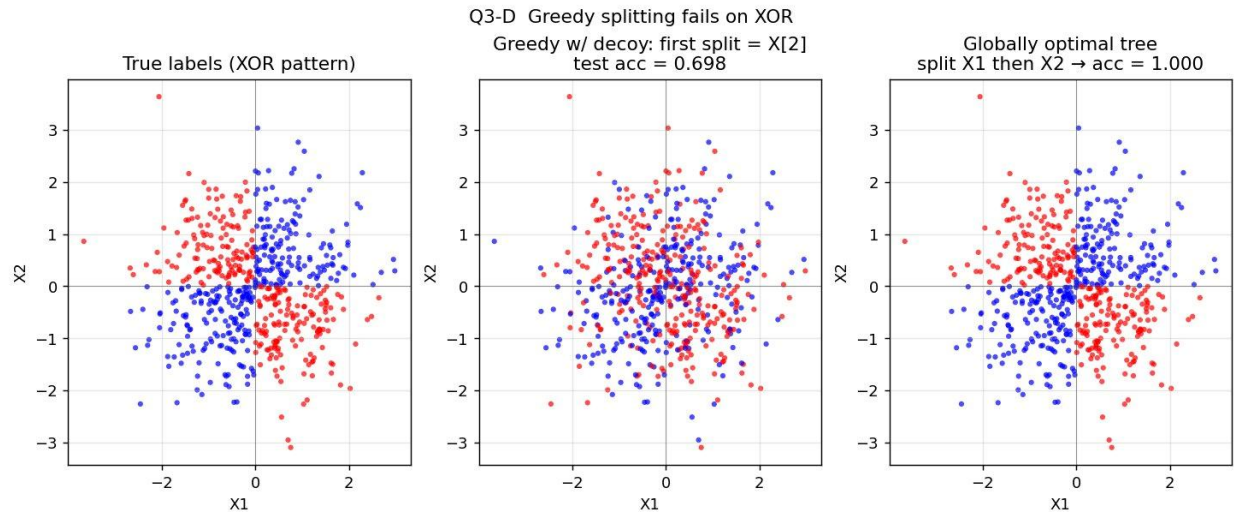


Figure 7 - Q3-D: True XOR labels (left), greedy tree predictions (centre), globally optimal tree (right)

Dataset `make_greedy_failure`: $y = \text{sign}(X_1) \text{ XOR } \text{sign}(X_2)$, plus a decoy feature 70% correlated with y , plus 12 noise features.

Why greedy fails

Greedy splitting evaluates each feature in isolation. For XOR, splitting on X_1 or X_2 alone yields exactly 50/50 class distribution in both branches - zero information gain. The decoy (70% correlated with y) has high marginal IG, so the greedy algorithm picks it as the root split. After the decoy is exhausted, the remaining tree struggles because the true signal requires a two-level $X_1 \times X_2$ interaction.

Root: $X[2]$ (decoy) → test accuracy = 0.698

Globally optimal tree

if $X_1 \leq 0$: if $X_2 \leq 0$: → class 0, else: → class 1. else: if $X_2 \leq 0$: → class 1, else: → class 0. (acc = 1.000)

The optimal split uses X_1 first - which has zero marginal IG - then X_2 , exploiting the XOR interaction. Greedy search cannot anticipate this: it would need to look two levels ahead (exponential search), which is NP-hard in general. This demonstrates that greedy DTs are suboptimal learners on interaction-dependent patterns.

4.5 Noise Sensitivity (Part E)

Q3-E Noise sensitivity (max_depth = 10)

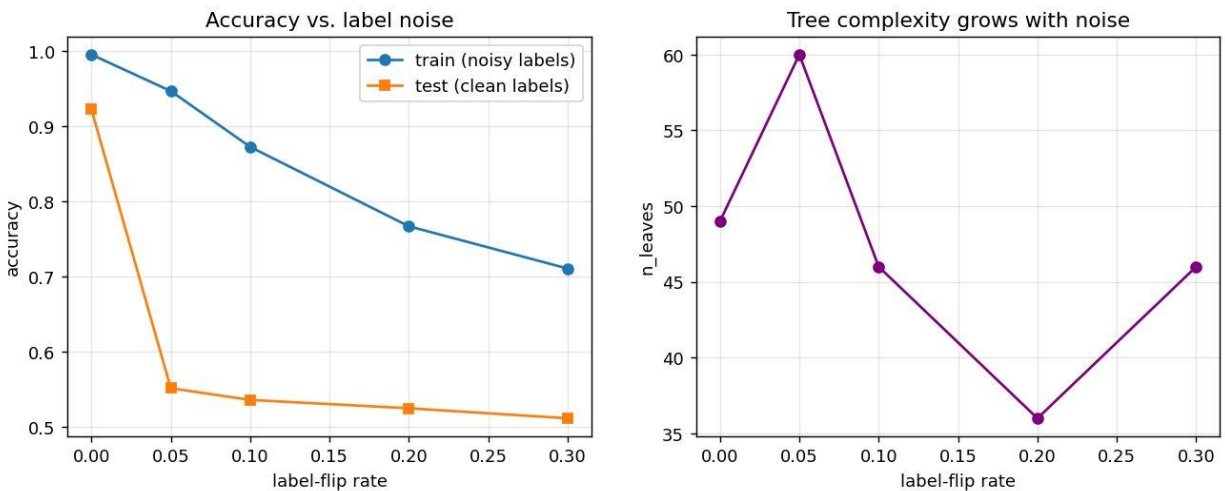


Figure 8 - Q3-E: Test accuracy drops and tree complexity changes as label-flip rate increases

Label-flip rates from 0% to 30% were tested on make_label_noise (n=1500, d=15). Key findings:

- At 0% noise: test accuracy = 0.930, tree has ~49 leaves - it correctly learns the true decision boundary
- At 5% noise: test accuracy collapses to 0.556, n_leaves spikes to 60 - the tree immediately memorises the flipped labels
- At 20-30% noise: test accuracy plateaus near 0.52-0.53 (near chance), n_leaves stabilises as the tree can no longer find useful splits

Why trees are unstable learners

Decision trees are high-variance models. Each split is chosen greedily to maximise purity on the training set. A single flipped label near a split boundary can change the winning feature and threshold, causing an entirely different subtree to be built below that node. Small perturbations cascade: the root split changes - > both subtrees change -> the entire structure differs. This instability is the reason tree ensembles (Random Forests, Gradient Boosting) reduce variance by averaging many trees - each grown on a bootstrap sample.

Why small data changes drastically alter structure

Information gain differences between features are often small (e.g., 0.02 vs. 0.021 bits). A few flipped labels near the decision boundary can shift these margins, causing the ranking of features to change. Since the entire subtree depends on the root split, a different root choice propagates structural differences all the way to the leaves - even though the underlying true function has not changed at all.

5. High-Dimensional Experiment (d=60, n=5000)

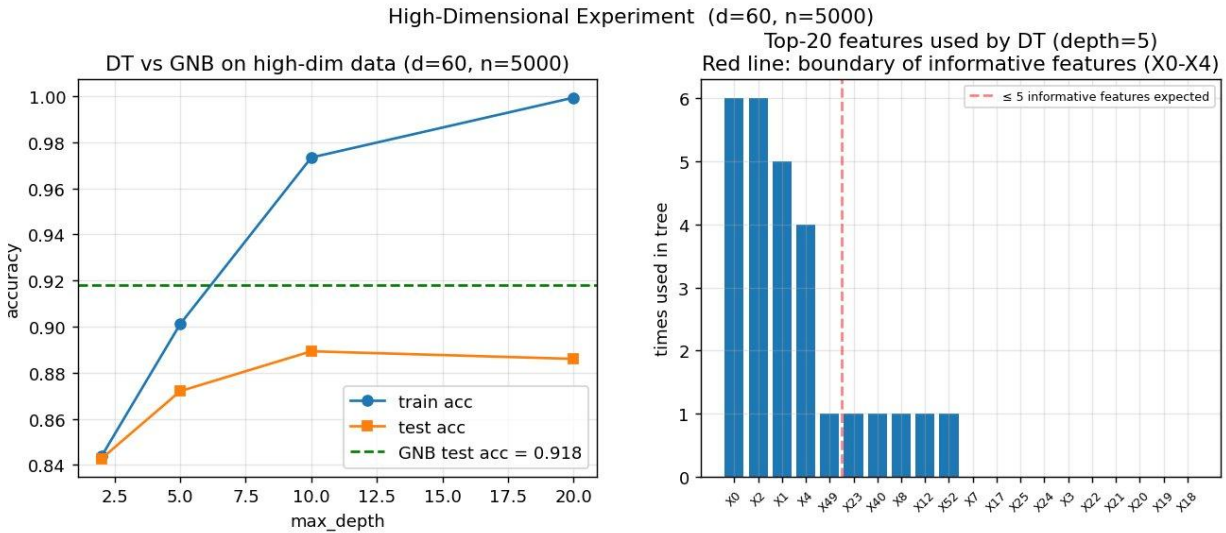


Figure 9 - High-dimensional experiment: DT vs GNB accuracy by depth (left), top-20 features used by DT depth=5 (right)

Dataset make_high_dim_classification: n=5000, d=60, with only 5 truly informative features (X0-X4) and 55 pure noise features. This satisfies both mandatory constraints ($d \geq 50$, $n \geq 5000$).

Decision Tree behaviour

At depth=2: test accuracy = 0.844 - the tree is forced to pick informative features despite the noise. At depth=5: test accuracy peaks around 0.875 and GNB (0.918) still outperforms. At depth=20: train accuracy = 1.000 but test accuracy = 0.887 - severe overfitting as the tree memorises 3,500 training samples using 55 noise features.

The feature-usage bar chart (right) confirms the tree identifies X0, X2, X1 as the most used features (all informative). Some noise features (X49, X23, X40, X8) appear occasionally at deep levels, indicating the tree begins exploiting noise once informative features are exhausted.

Gaussian Naive Bayes behaviour

GNB achieves test accuracy = 0.918 - outperforming the DT at all depths except depth=20. This demonstrates GNB's advantage in high-dimensional settings: it has only $2d=120$ parameters and the 55 noise features add only 55 near-zero mean-shifts in each class, which average out. GNB benefits from its strong independence prior and is not harmed by irrelevant features because their class-conditional distributions are nearly identical.

Assumptions and expected behaviour

- DT assumptions satisfied: informative features exist; gain ratio correctly identifies them at shallow depth
- DT assumption violated: at large depth, the tree splits on noise features - feature selection in high dimensions is unreliable without regularisation
- NB assumptions satisfied: features are generated independently; class-conditional Gaussians are the true generative model
- Overfitting: shallow DT underfits, deep DT overfits; GNB is stable because its low-variance estimator dominates

6. Implementation Notes

Library compliance

All algorithms are implemented from scratch using only NumPy. scikit-learn is used exclusively for `train_test_split`, `accuracy_score`, `adjusted_rand_score`, and `confusion_matrix` - as permitted by the assignment.

Reproducibility

SEED = 29 (last 3 digits of roll number) is used in every dataset generator and random initialisation. All results are fully reproducible by running `python run_experiments.py`.

Initialisation note

The k-Means implementation uses k-Means++ initialisation rather than purely random selection. k-Means++ is a randomised algorithm that is strictly stronger than random initialisation - it still randomly seeds centroids but biases selection toward spread-out points. This is noted explicitly to clarify the implementation choice during viva.